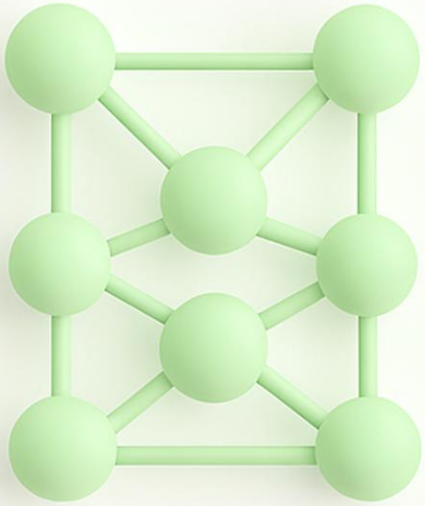


Deep Learning

Lakshmi Narasimhan

IIT Madras

Outline

1. Recap of Machine learning fundamentals
    - Linear algebra and probability
    - Supervised learning: Classification and Regression
    - Loss and errors
    - Model selection: over/under fitting, Bias-Variance tradeoff

Outline

2. Multilayer perceptron
    - Function approximation theorems
    - Feedforward neural networks
    - Activation functions
    - Width, depth and expressivity

Outline

3. Training neural networks

- Backpropagation
- Computational graph
- Optimization: Stochastic gradient descent and variants
- Avoiding overfitting: regularization, dropout, data augmentation
- Gradient instability, skip connection, initializations, early stop
- Normalization: input, layer, batch
- Hyperparameter tuning, curriculum training, transfer learning

Outline

4. Convolutional neural networks
    - Multi-channel inputs
    - Convolution and pooling
    - Padding, striding, flattening
    - Example architectures: AlexNet, GoogleNet, VGGNet, ResNet

Outline

5. Recurrent neural networks
   - Recursive neural networks
   - Backpropagation through time
   - Neural nets with memory: LSTM and GRU
   - Sequence learning and bidirectional LSTM
   - Optimizing RNN: Teacher forcing, leaky units, skipping through time

Outline

6. Attention mechanism
    - Self attention and multi-head attention
    - Query-key-value
    - Attention pooling and scoring
    - Embedding and encoding
    - Transformer architecture

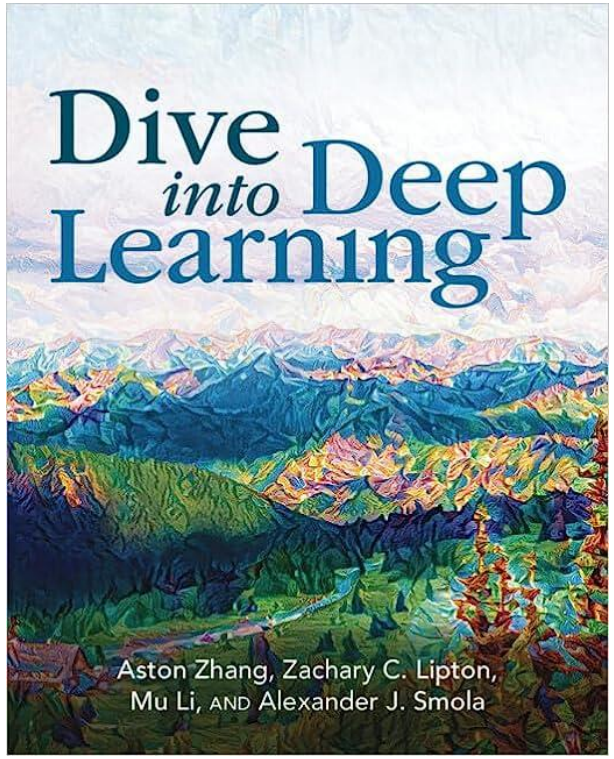
Outline

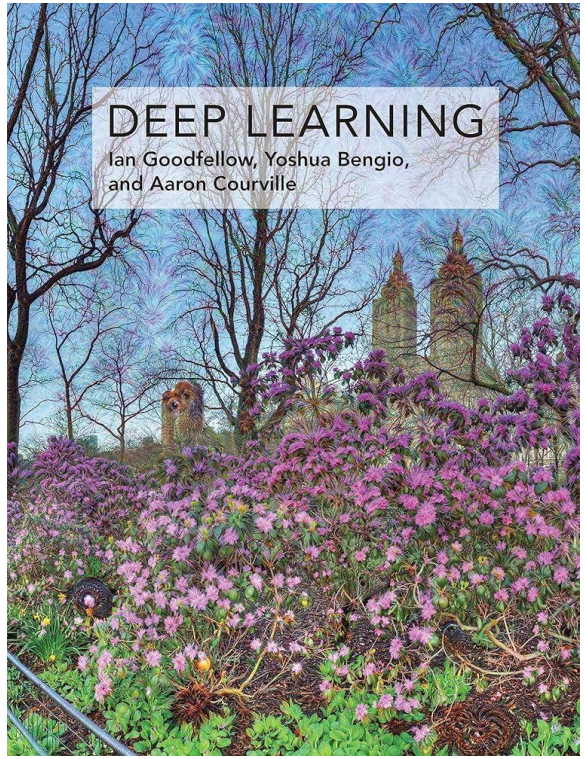
7. Autoencoders
    - Bottleneck architecture for encoder and decoders
    - PCA/sparse decomposition with autoencoders
    - Contractive and denoising autoencoders
    - Variational autoencoder

Outline

8. State-of-the-art
    - Graph neural networks
    - Copresheaf topological neural networks
    - Physics informed neural networks
    - Kolmogorov Arnold neural networks
    - Diffusion and generative models

Reference materials





Logistics

- Lectures: 5.30 PM to 7 PM on Mon & Thu (Jan 5th to Apr 10th)
- Tutorials: On alternate weeks
- Assignments: One per two weeks
  - Theory: 50%
  - Programming: 50%
- Head TAs
  - Manoj Kumar (da24s018@smail.iitm.ac.in)
  - Yuvaram Singh (da24s015@smail.iitm.ac.in)

Grading

- 25% - Mid term, 22 Feb
- 25% - End term, 26 Apr
- 20% - Assignments (4 best out of 6)
- 30% - Project
  - Finalize topic by Feb 28
  - Deadline: April 22
  - Submit: Report + codes

Sample Project Ideas

- Sentence Sentiment Classification
  - https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis
  - https://www.kaggle.com/datasets/kazanova/sentiment140
- Image Super Resolution
  - https://www.kaggle.com/datasets/adityachandrasekhar/image-super-resolution
  - https://www.kaggle.com/datasets/jesucristo/super-resolution-benchmarks
- GenAI with Variational Autoencoders
  - https://www.kaggle.com/code/basu369victor/generate-music-with-variational-autoencoder
  - https://www.kaggle.com/code/averkij/variational-autoencoder-and-faces-generation
- Product Recommender System
  - https://www.kaggle.com/datasets/shivamb/fashion-clothing-products-catalog
  - https://www.kaggle.com/datasets/lokeshparab/amazon-products-dataset

Project Report Format

- Introduction (1 mark)
- Background and state-of-the-art (1 mark)
- Problem Statement (4 marks)
- Dataset Description (2 marks)
- Solution/Methodology/Neural Net Architecture (4 marks)
- Theoretical Analysis (2 marks)
- Data Preprocessing and Training Methodology (3 marks)
- Experimental Results (5 marks for plots & description + 5 marks for attached code)
- Inferences/Insights Obtained/Difficulties faced (3 marks)
- Template - https://www.overleaf.com/read/cbbbwmpqctbv#397b3a

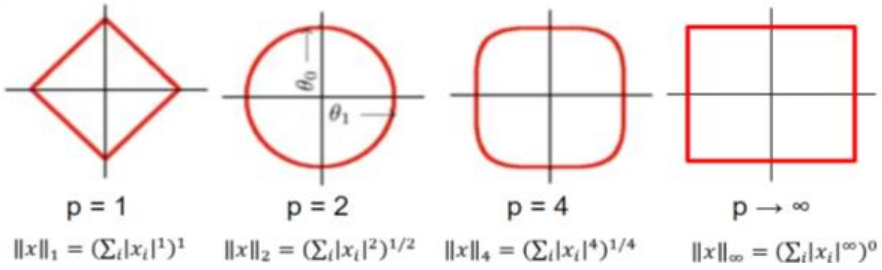
Linear algebra and probability

Vectorspaces

- A set V : $\forall \mathbf{x}, \mathbf{y} \in V$, $a\mathbf{x} + b\mathbf{y} \in V$, where a and b are any scalars
  - Closed under addition and scalar multiplication
- Subspace: a subset of V that is closed under addition and scalar multiplication
- Any element of this vectorspace is a vector
  - For finite-dimensional vectorspaces (e.g., $\mathbb{R}^n$): n x 1 array of numbers
  - In our study, most data samples will be represented by a vector

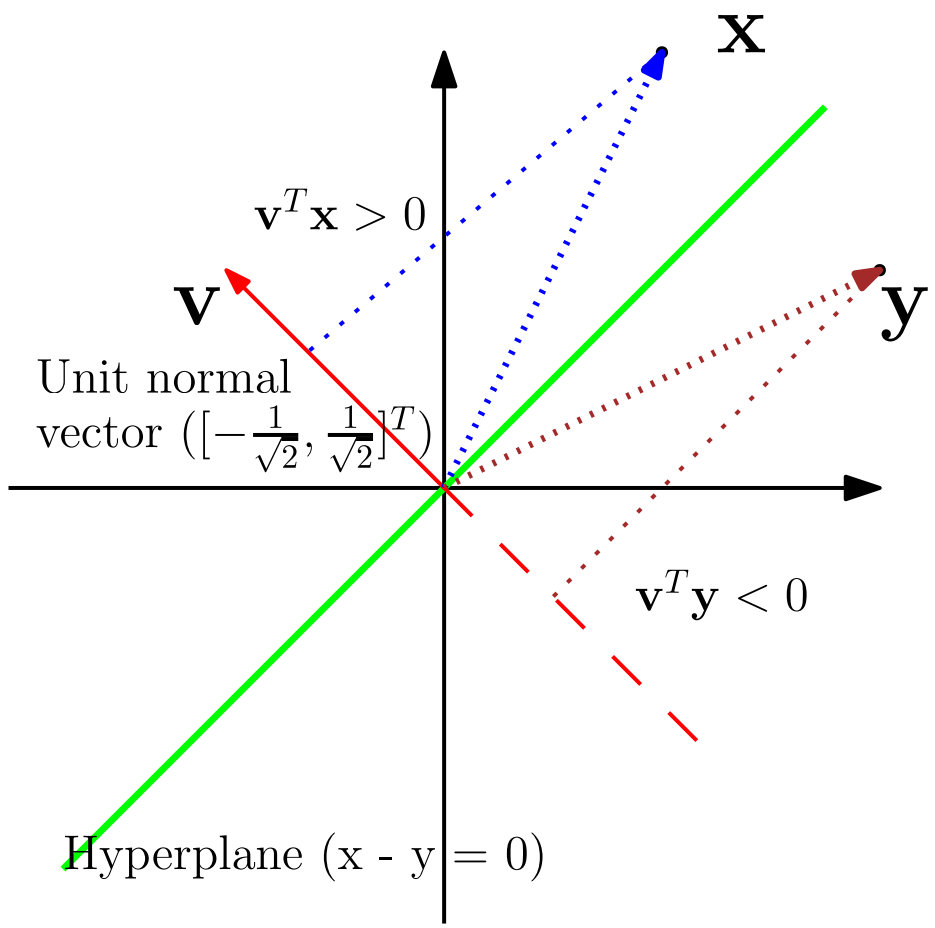
Orthogonality & Norms

- An inner product on $\mathbb{R}^n$: $\mathbf{x}^T\mathbf{y}$
  - Measure of component of $\mathbf{x}$ on $\mathbf{y}$ (and vice versa)
  - Orthogonal: $\mathbf{x}^T\mathbf{y} = 0$ (perpendicular vectors)
  - $\mathbf{x}^T\mathbf{y} > 0$: $\mathbf{x}$ and $\mathbf{y}$ have components along the same direction
  - $\mathbf{x}^T\mathbf{y} < 0$: $\mathbf{x}$ and $\mathbf{y}$ have components along opposite directions
- p - norm ($p \geq 1$) : $\|\mathbf{x}\|_p := \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}$
  - Euclidean norm: $\|\mathbf{x}\|^2 = \mathbf{x}^T\mathbf{x}$



Hyperplanes

- Linear hyperplane
  - An *n - 1* dimensional subspace of $\mathbb{R}^n$
  - Passes through the origin
  - Divides $\mathbb{R}^n$ into two halves
  - Represented by a normal vector
- Normal vector:
  - A vector orthogonal to all vectors on the hyperplane
  - Distance of $\mathbf{x}$ from hyperplane = $\mathbf{x}^T\mathbf{v} / \|\mathbf{v}\|$



Singular Value Decomposition

- Singular value decompositon : $\mathbf{M} = \mathbf{U}\Sigma\mathbf{V}^T$
  - Exists for any matrix
  - **U**, **V** are orthogonal: left and right singular vectors
  - Σ is PSD diagonal: The diagonal elements are singular values
  - The diagonal elements of Σ are in ascending order : $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_n \geq 0$
- Pseudoinverse: $\mathbf{V}\Sigma_r^{-1}\mathbf{U}^T$, where r is the rank of **M**
- Matrix norm: $\|A\|_p = \sup_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}$ $\qquad \|A\|_2 = \sigma_{\max}(A)$

Random Variable

- Almost all data collected in real life are affected by noise
  - This measurement is an outcome of the data collection experiment
  - The outcome may change every time the experiment is performed
  - However, certain *statistical* properties of these outcomes are deterministic
- Probability space: (Ω, F, P) – quantifying randomness of measurements
  - Ω : Sample space; F : Event space – all possible subsets of Ω
  - P : a measure for all elements of F – e.g., proportion of occurrence of any $u \in F$
- Random variable: a mapping from Ω to $\mathbb{R}$ (real numbers)

PMF vs PDF

Discrete random variables	Continuous random variables
Ω is countable all $w \in \Omega$ has $0 \le P(w) \le 1$	Ω is uncountable $u \in F$ has $0 \le P(u) \le 1$, $w \in \Omega$ has $P(w) = 0$
$X(w)$ is a real-discrete-valued r.v. $P(w)$ is its probability *mass*	$X(w)$ is a real-continuous-valued r.v. $P(w)$ is its probability *density*
Probability mass function (PMF) $P(X = x) = P(w)$ for $X(w) = x$	Probability density function (PDF) $P(X \in [a,b]) = \int_a^b P(x)dx = P(u)$, $u = \{X^{-1}(x), x \in [a,b]\}$
Law of total probability $\Sigma_i P(X = x_i) = P(\Omega) = 1$	Law of total probability $\int P(x)dx = P(\Omega) = 1$

Conditional Probability & Independence

- $P(X \mid Y)$: probability of occurrence of X when Y has occurred
  - The sample space is reduced by conditioning
    - Creates a new probability space and a valid PMF/PDF – $P(X = x \mid Y)$
    - Probability can increase or decrease upon conditioning
  - $P(X \mid Y) = P(X, Y)/P(Y)$
    - $P(X \mid Y = y)$ is a function of x, but $P(X \mid Y)$ is a function of both x and y
- Independent r.v.s: X and Y do not influence each other
  - $P(X, Y) = P(X)\,P(Y) \implies P(X \mid Y) = P(X)$

Bayes Theorem

- $P(X \mid Y) = P(Y \mid X)\, P(X)/P(Y)$
  - Given that Y is observed and a dependent hidden variable X exists, we can infer the probability of the hidden variable
- $P(X)$: Prior, statistics known before any observation
- $P(X \mid Y)$: Posterior, computed indirectly after observing Y
  - $P(X \mid Y = y)$ is a valid PMF/PDF; $P(X \mid Y = y) \propto P(Y = y \mid X)\, P(X)$
- $P(Y \mid X)$: Likelihood, computed based on the outcome of Y
  - $P(Y = y \mid X)$ is not a valid PMF/PDF
    - Note that the probability space itself changes for each value of X

Gaussian Distribution

- Gaussian $X \sim \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{\sigma^2}}$
  - μ, σ : Only two parameters describe the entire PDF
- Multivariate Gaussian: $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} e^{-(\mathbf{x}-\mu)^T \Sigma^{-1} (\mathbf{x}-\mu)}$
  - $\mathbf{x} = [X_1, \ldots, X_n]^T$ has n Gaussian r.v.'s that are also jointly Gaussian
  - $\boldsymbol{\mu} = \mathbb{E}\{\mathbf{x}\}$ is the mean vector, $\boldsymbol{\Sigma} = \mathbb{E}\{(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T\}$ is a PSD covariance matrix
  - If $\mathbf{z} = \boldsymbol{\Sigma}^{-½}\mathbf{x}$, then the elements of $\mathbf{z}$ are independent Gaussian r.v.
  - Precision parameter: $\boldsymbol{\beta} = \boldsymbol{\Sigma}^{-1}$

KL Divergence

- Maximum entropy distributions (priors with least bias)
  - Uniform for discrete r.v.
  - Gaussian for continuous r.v.
- Relative entropy: $D(P \| Q) = \mathbb{E}_P\{\log P/Q\} \geq 0$
  - Measure of how distant is the distribution Q is different from P
- Cross entropy: $H(P; Q) = \mathbb{E}_P\{ - \log Q\} \geq 0$ (for discrete r.v.)
  - $H(P; Q) = D(P \| Q) + H(P)$
  - Minimizing CE is equivalent to minimizing KLD

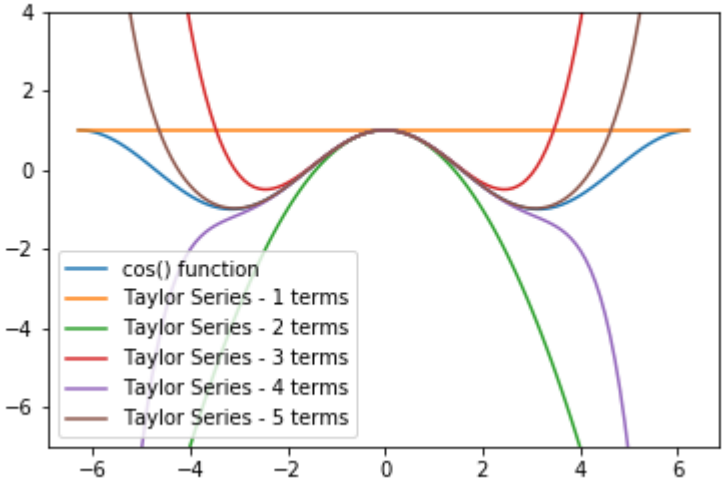
Classical vs Bayesian

Bayesian	Classical
Parameter to be estimated is a random variable	Parameter to be estimated is an unknown deterministic quantity
A prior distribution on the parameters is used	No prior (all values are equally likely - special case of Bayesian)
Posterior probability is optimized	Likelihood is optimized
Poor performance when prior is incorrect	Poor performance whenever the parameter is not uniformly distributed

Taylor Expansion

- Polynomial approximation

$$T(\mathbf{x}) = f(\mathbf{a}) + (\mathbf{x}-\mathbf{a})^{\mathsf{T}} Df(\mathbf{a}) + \frac{1}{2!}(\mathbf{x}-\mathbf{a})^{\mathsf{T}} \left\{D^2 f(\mathbf{a})\right\} (\mathbf{x}-\mathbf{a}) + \cdots$$

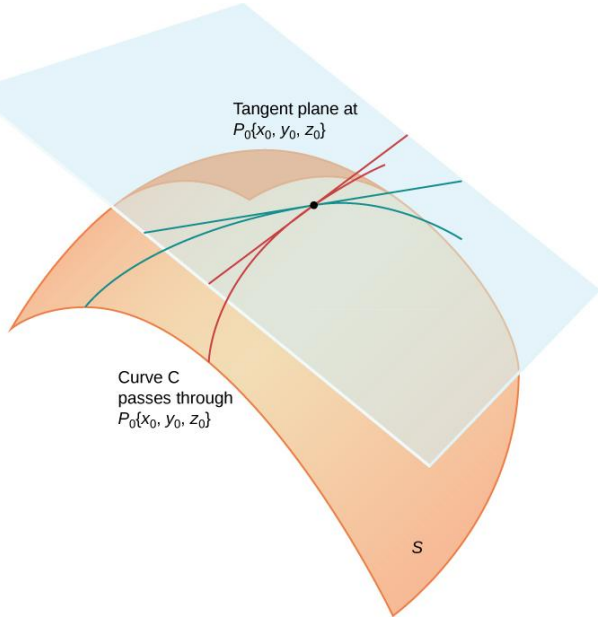


- (

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

essia

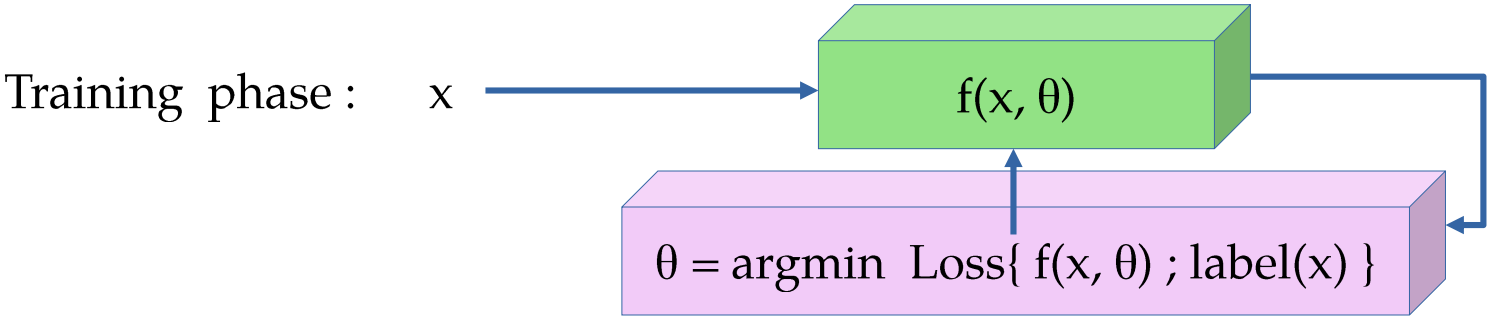
$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1\, \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1\, \partial x_n} \\ \frac{\partial^2 f}{\partial x_2\, \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2\, \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n\, \partial x_1} & \frac{\partial^2 f}{\partial x_n\, \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$



Machine Learning Fundamentals

Supervised/Unsupervised Learning

Supervised Learning	**Unsupervised Learning**
Training + Testing (inference) phase	Only inference phase
Requires labeled data for training	All data are unlabeled
Examples: Classification & estimation	Clustering, PCA, pattern finding



Loss Functions

- How to find the best parameters while training a supervised learning model?
  - Minimize the error between model output for input x_i and the label of x_i
- Loss function: A quantification of the error obtained during parameter optimization $\theta = \text{argmin}\ \text{Loss}\{ f(x, \theta) ; \text{label}(x) \}$
  - L0 norm (sparsity-promoting), 0-1 loss: $\| y - y_{true} \|_0$
  - L1 norm, absolute error: $\| y - y_{true} \|_1$
  - Squared L2 norm, Euclidean distance: $\| y - y_{true} \|^2_2$
  - Mixed norms: linear combination of different p-norms ($p \geq 1$) – convex losses
  - Cross entropy, F-score, etc.

Regression

- Setup: $y_i = f(x_i) + \varepsilon$
- Find: f(.). Parameterized $f(x) = g(\mathbf{w}, x)$; linear regression: $g(\mathbf{w}, \mathbf{x}) = \mathbf{x}^T\mathbf{W}$
- Given (training data)
  - Labels: $\{y_i\}$
  - Inputs : $\{x_i\}$
- Predict (testing data)
  - Compute {y} for unseen {x}

Classification

- Setup: $x_{i,j} \in C_i$; $i = 1, \ldots, K$ and $j = 1, \ldots, N$
- Find: $f(.)$ s.t. $f(x_{i,j}) = C_i$
- Given (training data)
  - Labels: $\{y_i\}$ s.t. $y_i \in \{C_i\}$
  - Inputs : $\{x_{i,j}\}$
- Predict (testing data)
  - Compute $\{y\}$ for unseen $\{x\}$ s.t. $y \in \{C_i\}$

Classification

- Binary classification: K = 2, C_1 = +1 and C_2 = -1
- Posterior probability for binary classification is

$$\Pr(y_i = +1|x_i) = \frac{\Pr(x_i|y_i = +1)\Pr(y_i = +1)}{\Pr(x_i|y_i = +1)\Pr(y_i = +1) + \Pr(x_i|y_i = -1)\Pr(y_i = -1)}$$
$$= \frac{1}{1 + e^{-a}} = \sigma(a); \quad a = \ln \frac{\Pr(x_i|y_i = +1)\Pr(y_i = +1)}{\Pr(x_i|y_i = -1)\Pr(y_i = -1)}$$

a = log posterior ratios

- The sigmoid function naturally arises here

Classification

- Posterior probability for K-class classification is

$$\Pr(y_i = C_k|x_i) = \frac{\Pr(x_i|y_i = C_k)\Pr(y_i = C_k)}{\sum_j \Pr(x_i|y_i = C_j)\Pr(y_i = C_j)}$$
$$= \frac{e^{a_k}}{\sum_j e^{a_j}} = \sigma_k(\mathbf{a}); \quad a_k = \ln \Pr(x_i|y_i = C_k)\Pr(y_i = C_k)$$

- The softmax function naturally arises here

Inference on Gaussian Data

- Binary classification

$$\Pr(\mathbf{x}_i|y_i=1) = \frac{1}{(2\pi)^{M/2}\det(\mathbf{\Sigma})^{1/2}} e^{-\frac{(\mathbf{x}_i-\boldsymbol{\mu}_k)^T\mathbf{\Sigma}^{-1}(\mathbf{x}_i-\boldsymbol{\mu}_k)}{2}}$$

$$\Pr(y_i=+1|\mathbf{x}_i) = \sigma(\mathbf{x}_i^T\mathbf{w}+w_0);\ \mathbf{w}=\mathbf{\Sigma}^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_{-1})$$

$$w_0 = -\frac{\boldsymbol{\mu}_1^T\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_1}{2} + \frac{\boldsymbol{\mu}_{-1}^T\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_{-1}}{2} + \ln\frac{\Pr(y_i=1)}{\Pr(y_i=-1)}$$

- Multiclass classification

$$\Pr(y_i=+1|\mathbf{x}_i) = \sigma_k(\mathbf{x}_i^T\mathbf{W}+\mathbf{w}_0);\ \mathbf{W}=\mathbf{\Sigma}^{-1}[\boldsymbol{\mu}_1,\cdots,\boldsymbol{\mu}_K]$$

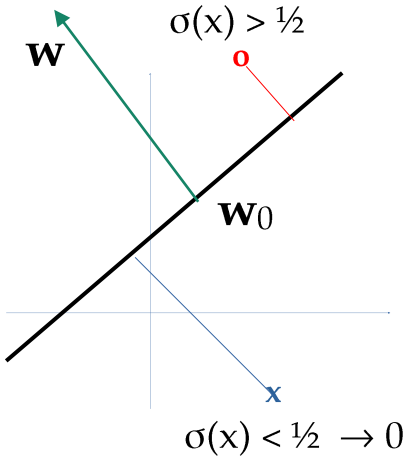
$$\mathbf{w}_0 = [-\frac{\boldsymbol{\mu}_1^T\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_1}{2} + \ln\Pr(y_i=1),\cdots,-\frac{\boldsymbol{\mu}_K^T\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_K}{2} + \ln\Pr(y_i=K)]$$

- Ridge regression

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \arg\min_{\boldsymbol{\theta}} \|\mathbf{y}-\mathbf{G}\boldsymbol{\theta}\|_2^2 \quad \text{s.t.} \quad \|\boldsymbol{\theta}\|_2^2 \le B \\ &= \arg\min_{\boldsymbol{\theta}} \|\mathbf{y}-\mathbf{G}\boldsymbol{\theta}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_2^2 \\ &= (\lambda I + \mathbf{G}^T\mathbf{G})^{-1}\mathbf{G}^T\mathbf{y}\end{aligned}$$

Classifying Linearly Separable Data

- Hyperplane projection: $(\mathbf{x} - \mathbf{w}_0)^T\mathbf{w} = \mathbf{x}^T\mathbf{w} - \mathbf{w}_0^T\mathbf{w} = \mathbf{x}^T\mathbf{w} - w_0$
  - $\mathbf{w}_0$ is the shift of origin and $\mathbf{w}$ is normal to the hyperplane
- Binary classification
  - When x is on the hyperplane, $\Pr(y = C_1) = \sigma(0) = ½$
  - Farther from the hyperplane $\sigma(\text{ +ve }) \to 1$ or $\sigma(\text{ -ve }) \to 0$
  - Sigmoid output gives the PMF of the Bernoulli r.v. f(x)
- Multi-class classification
  - When x is on the hyperplane, $\Pr(y = C_1) = \sigma_k(\mathbf{0}) = 1/K$
  - Farther from the k^{th} hyperplane $\sigma_k(\text{ +ve }) \to 1$ or $\sigma_k(\text{ -ve }) \to 0$
  - Softmax output gives the PMF of the categorical r.v. f(x)



Logistic Regression – Single Layer Neural Net

- Let the classes be one-hot encoded (to represent a PMF)

$$\Pr(\mathbf{y}|\mathbf{x},\mathbf{W}) = \prod_{k=1}^{K} \Pr(y = C_i|\mathbf{x},\mathbf{W})^{y_k}; \qquad \sum_{k=1}^{K} y_k = 1; \qquad \sum_{k=1}^{K} \Pr(y = C_k|\mathbf{x},\mathbf{W}) = 1$$

- To find $\mathbf{w}$: do maximum likelihood regression

$$\arg\min_{\mathbf{W}} -\ln \Pr(\mathbf{y}|\mathbf{x},\mathbf{W}) = \arg\min_{\mathbf{W}} -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{i,k} \ln \sigma_k(\mathbf{x}_i^T\mathbf{W}) = \arg\min_{\mathbf{W}} \sum_{i} H(y_i; \sigma_k(\mathbf{x}_i^T\mathbf{W}))$$

- The optimal hyperplanes are obtained using gradient descent

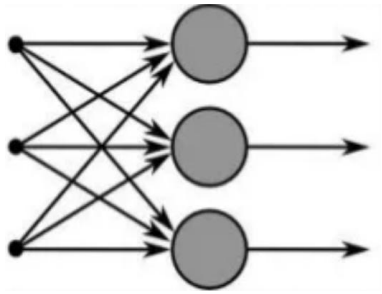
$$\mathbf{w}_{j,t+1} = \mathbf{w}_{j,t} - \eta\nabla_{\mathbf{w}_j}\mathcal{L} = \mathbf{w}_{j,t} - \eta\sum_{i}(\sigma_j(\mathbf{x}_i^T\mathbf{W}_t) - y_{i,j})\mathbf{x_i}$$

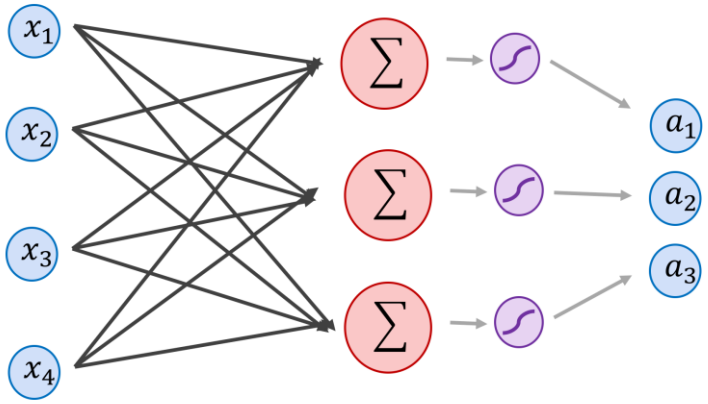
Summary

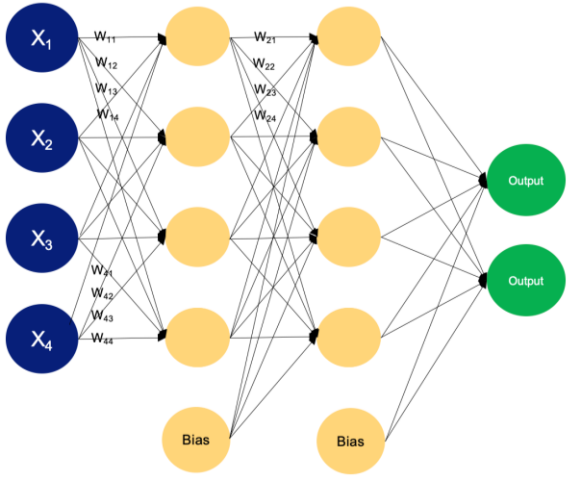
- We focus on parametric supervised learning framework
- Neural models can be viewed as interpolators or curve-fitters
- Loss function: measure of deviation from expected output/label
- Optimal linear/ridge regression: a single layer linear neural net
- Optimal linear classification: a single layer neural net with sigmoid/softmax activation
- Learning weights from training data implicitly learns data prior

Extending to Multiple Layers

- We cannot model XOR with a linear classifier
- What if cascade multiple single layer neural net?
  - Similar to making intermediate decisions
- Can we keep increasing the number of layers/parameters?

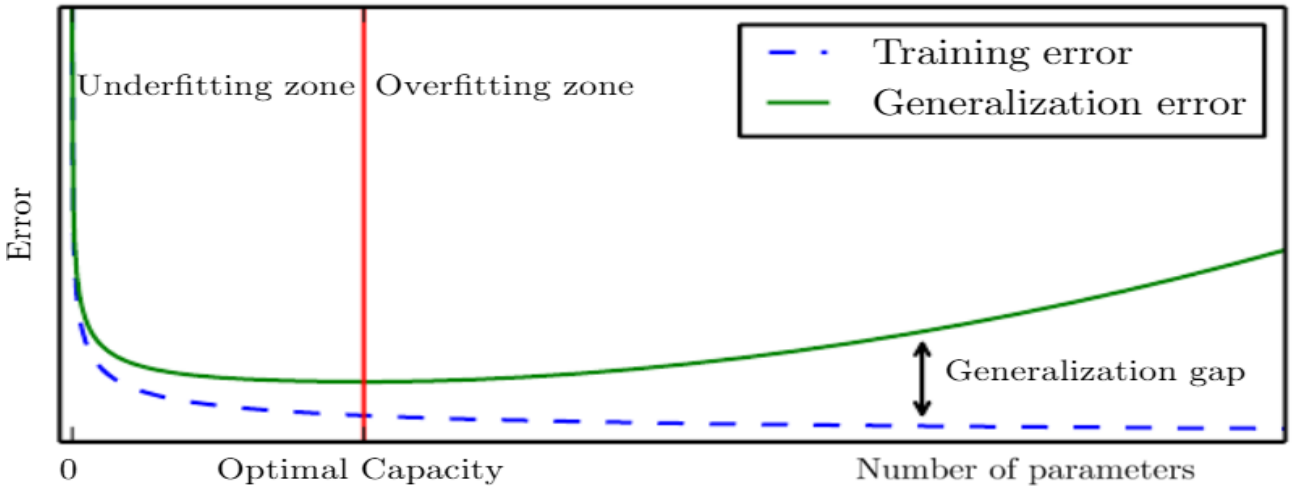


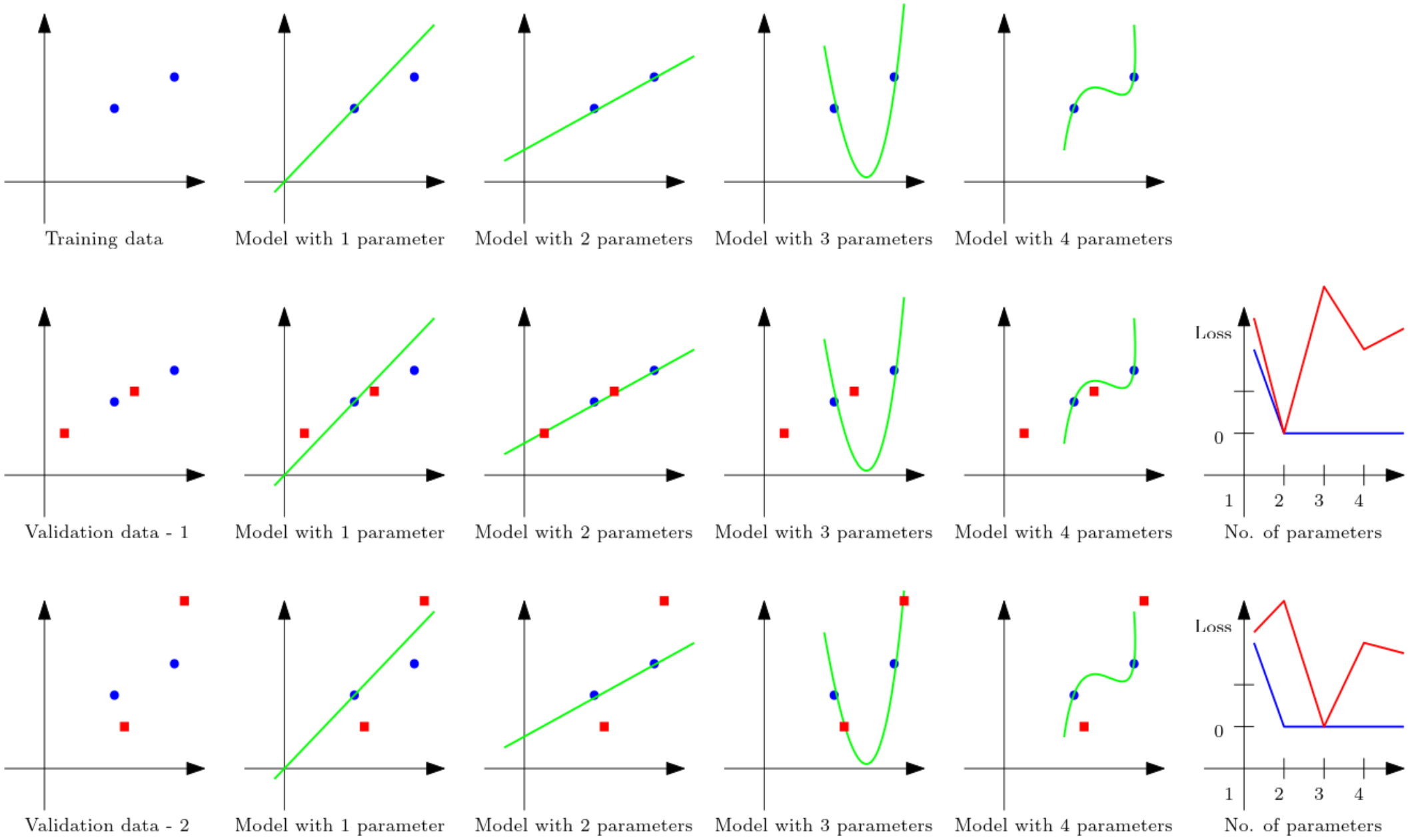




Training/Generalization Error

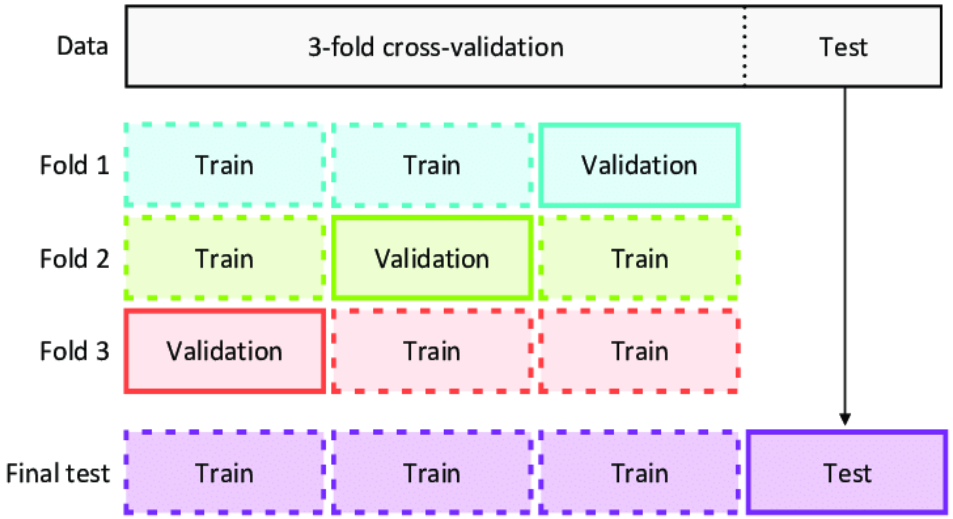
- Increasing number of parameters is not always good
  - Training error may go down, but testing (generalization) error may be high
- There is an optimal number of parameters for a given kind of data
  - VC dimension (| Test error - Training error | $\leq$ $O(d_{VC} \, log \, n/n)^{1/2}$)
- Underfitting: Less than optimal
- Overfitting: More than optimal
  - How to identify overfitting?




Training data
Model with 1 parameter
Model with 2 parameters
Model with 3 parameters
Model with 4 parameters
Validation data - 1
Model with 1 parameter
Model with 2 parameters
Model with 3 parameters
Model with 4 parameters
Loss
0
1
2
3
4
No. of parameters
Validation data - 2
Model with 1 parameter
Model with 2 parameters
Model with 3 parameters
Model with 4 parameters
Loss
0
1
2
3
4
No. of parameters

Validation Error & Overfitting

- Split available labeled data to training + validation sets
  - Sequentially minimize both training and validation errors
  - Use validation set to identify overfitting
- Validation error: The error measured with validation set – An indicator of generalization error

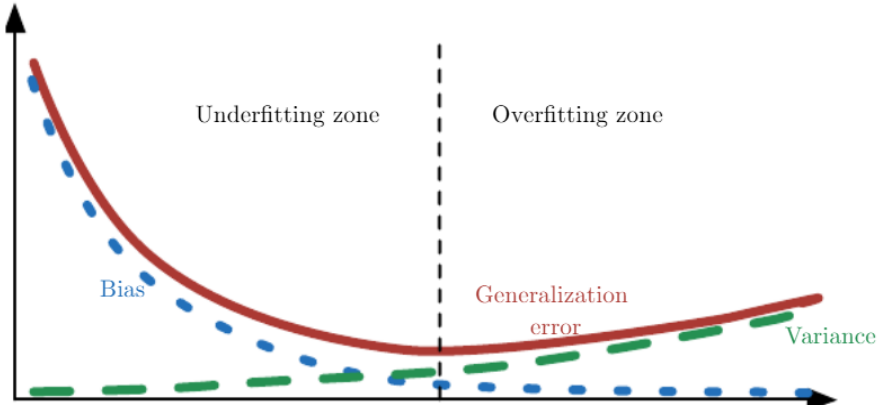


- Cross validation: When very few labeled data is available
  - Use different training+validation split in each iteration of training

Bias-Variance & Model-fitting

- Bias: E{y – y_{true}} – average deviation from true value
- Variance: Var{y} – average deviation of inference
- Is lower bias or variance always good?

$$\begin{aligned}\mathbb{E}((y-\hat{y})^2) &= \mathbb{E}(y^2) + \mathbb{E}(\hat{y}^2) - 2\mathbb{E}(\hat{y})\mathbb{E}(y) \\ &= \mathbb{E}(y^2) - \mathbb{E}(y)^2 + \mathbb{E}(\hat{y}^2) - \mathbb{E}(\hat{y})^2 - 2\mathbb{E}(\hat{y})\mathbb{E}(y) + \mathbb{E}(y)^2 + \mathbb{E}(\hat{y})^2 \\ &= Var(y) + Var(\hat{y}) + \text{Bias}^2\end{aligned}$$



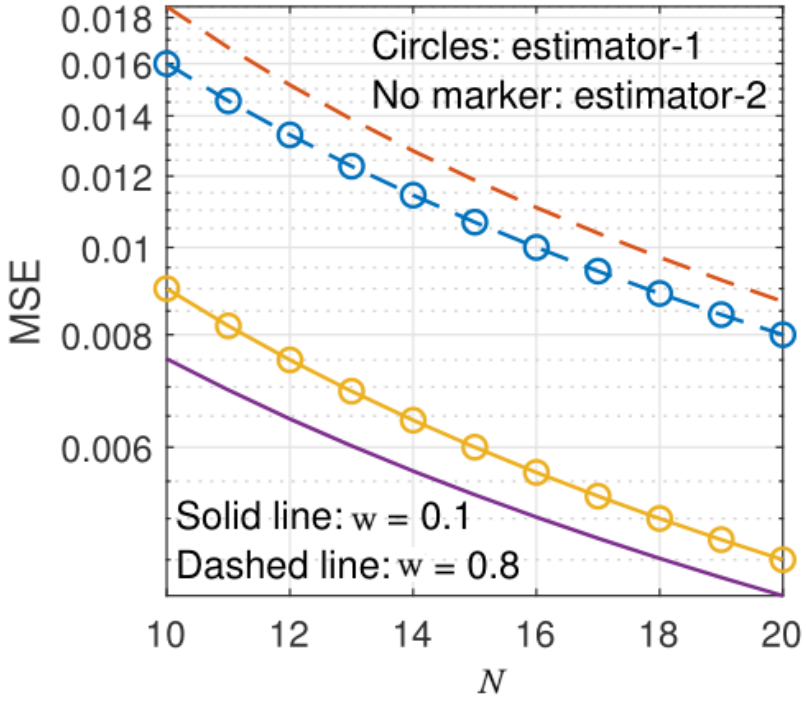
- Overfitting : High model variance and low MSE in training
- Underfitting : Low model variance and high MSE in training

Bayesian Inference

- Infer bias (p) of a coin from observations of its toss: $X_i \sim Be(p)$
- (Classical) No bias

$$\hat{p}_1 = \arg\max_p p^{\sum y_i}(1-p)^{N-\sum y_i} = \frac{\sum y_i}{N}$$

$$Var(\hat{p}_1) = \frac{p(1-p)}{N} \quad \text{Bias}(\hat{p}_1) = 0$$



$$\hat{p}_3 = \arg\max_p p^{\sum y_i}(1-p)^{N-\sum y_i} P(p) = \frac{\sum y_i}{N+1}$$

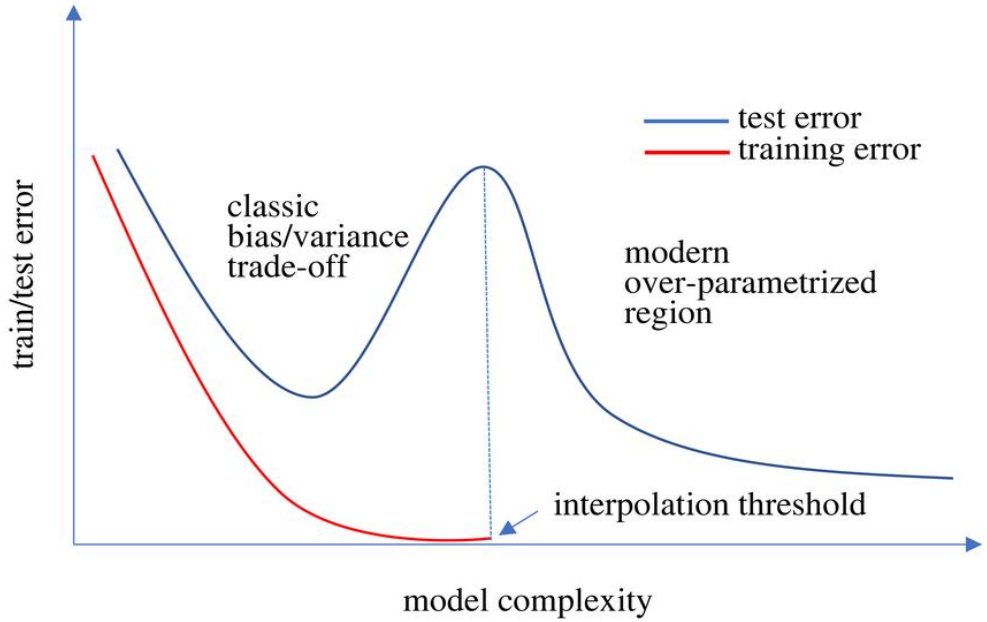
- (Bayesian) Bias is already in-built in prior

No Free-Lunch Theorem

- Training dataset may not contain all possible samples of input
  - How would the unseen data affect the learned statistics?
- No free-lunch theorem: As the unseen data can have different statistics, there can exist a dataset where any model (optimally trained with a training dataset) can perform poorly
  - So, will all models perform equally bad on average?
  - Tradeoff between robustness and accuracy
- Inductive Bias: If some statistics of the entire data is known, use that information in addition to training data to optimize the model – *different architectures*

Double Descent

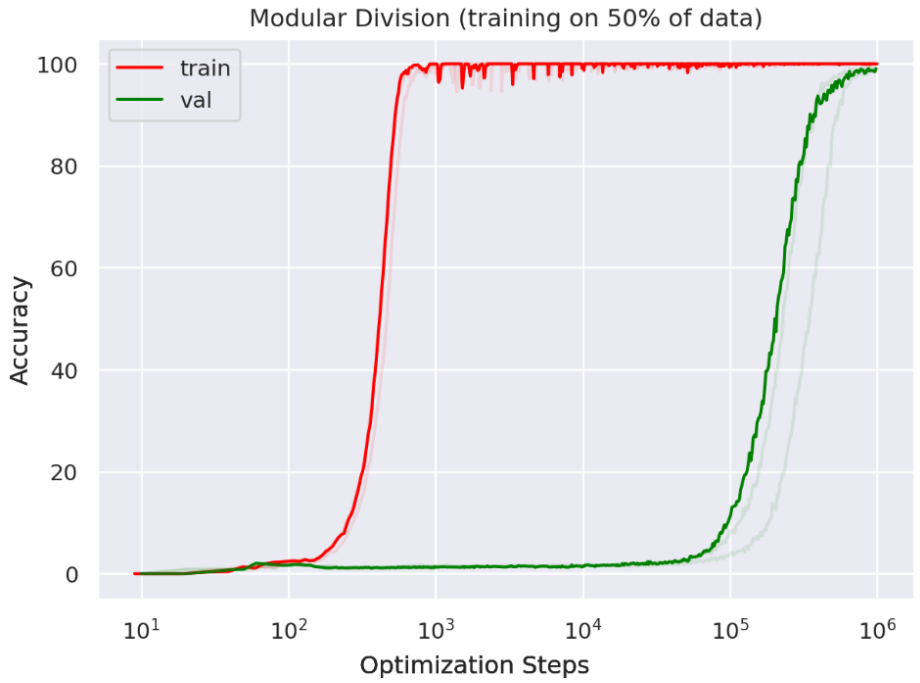
- Overfitting can be overcome
  - With deeper networks
- As the number of parameters (depth) increases, the overfitting is smoothened
  - Smoother interpolations and extrapolations



- Interpolation threshold
  - Where the number of parameters equals the number of data points
  - R. Shaeffer et al, *Double Descent Demystified: Identifying, Interpreting & Ablating the Sources of a Deep Learning Puzzle*
  - Some times more training data can hurt performance!

Grokking

- Generalization of over-parameterized nets by over training
- Amount of over training reduces with decrease in dataset size
  - *Grokking Explained: A Statistical Phenomenon*, 2025
- Weight decay helps
  - Additional term in the loss function to push the weights closer to zero



Assignment

- Generate training 100 samples : $y = x + 0.3x^2 + 0.7x^3 + w$
  - x is uniformly distributed in [-3 to 3]
  - w is Gaussian noise with mean 0 and standard deviation 0.09
- Perform linear regression for model sizes M = 1 to 120 and plot
  - Bias (average of y – f(x)) vs M
  - Variance of y vs M
  - Training error vs M
  - Validation error vs M

Multilayer Perceptron

Extending Single-Layer Neural Nets

- For linear data models
  - $y = \mathbf{w}^T\mathbf{x}$ or $\sigma(\mathbf{w}^T\mathbf{x} + b)$
  - Perceptron: $u(\mathbf{w}^T\mathbf{x} + b)$
- In general, $y = f(\mathbf{w}, \mathbf{x})$
  - What are some good parametric functions?
  - Are there a family of such functions that can fit any arbitrary N-dimensional curve?

Universal Function Approximation Theorems

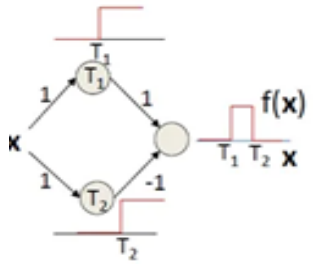
- Stone - Weierstrass (1885)
  - Every continuous function can be approximated by polynomials
- Andrew Barron (1993 - Universal Approximation Bounds for Superpositions)
  - Lipshitz functions can be approximated by infinite-width 1-layer NN
- Cybenko (1989 - Approximation by superpositions of a sigmoidal function)
  - Lipshitz functions can be approximated by infinite-width 1-layer sigmoid NN

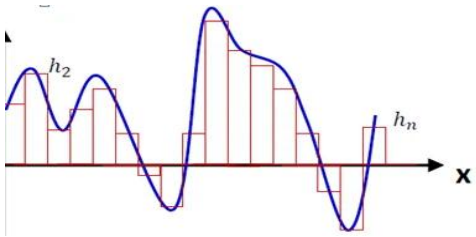
Universal Function Approximation Theorems

- Hornik – Stinchcombe – White (1989 - Multilayer feedforward networks are universal approximators)
  - Lipshitz functions can be approximated by finite-width multi-layer NN
- Lu – Pu – Wang – Hu – Wang (2017 - The expressive power of neural network)
  - Lipshitz functions can be approximated by finite-width multi-layer ReLU NN
- Kolmogorov–Arnold (1957)
  - Lipshitz functions can be approximated by 2-layer NN

Universal Function Approximation

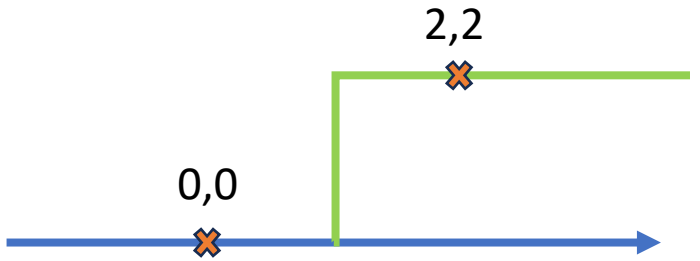
- Polynomials in high dimensions : very high complexity
- Any function f(x) (with some regularity conditions) can be approximated by a superposition of scaled and shifted step functions
  - $f(x) = \Sigma\, a_i\, u(w_i\, x - b_i)$
  - w_i & b_i are the scale and shift factors
  - $u()$ is the activation function



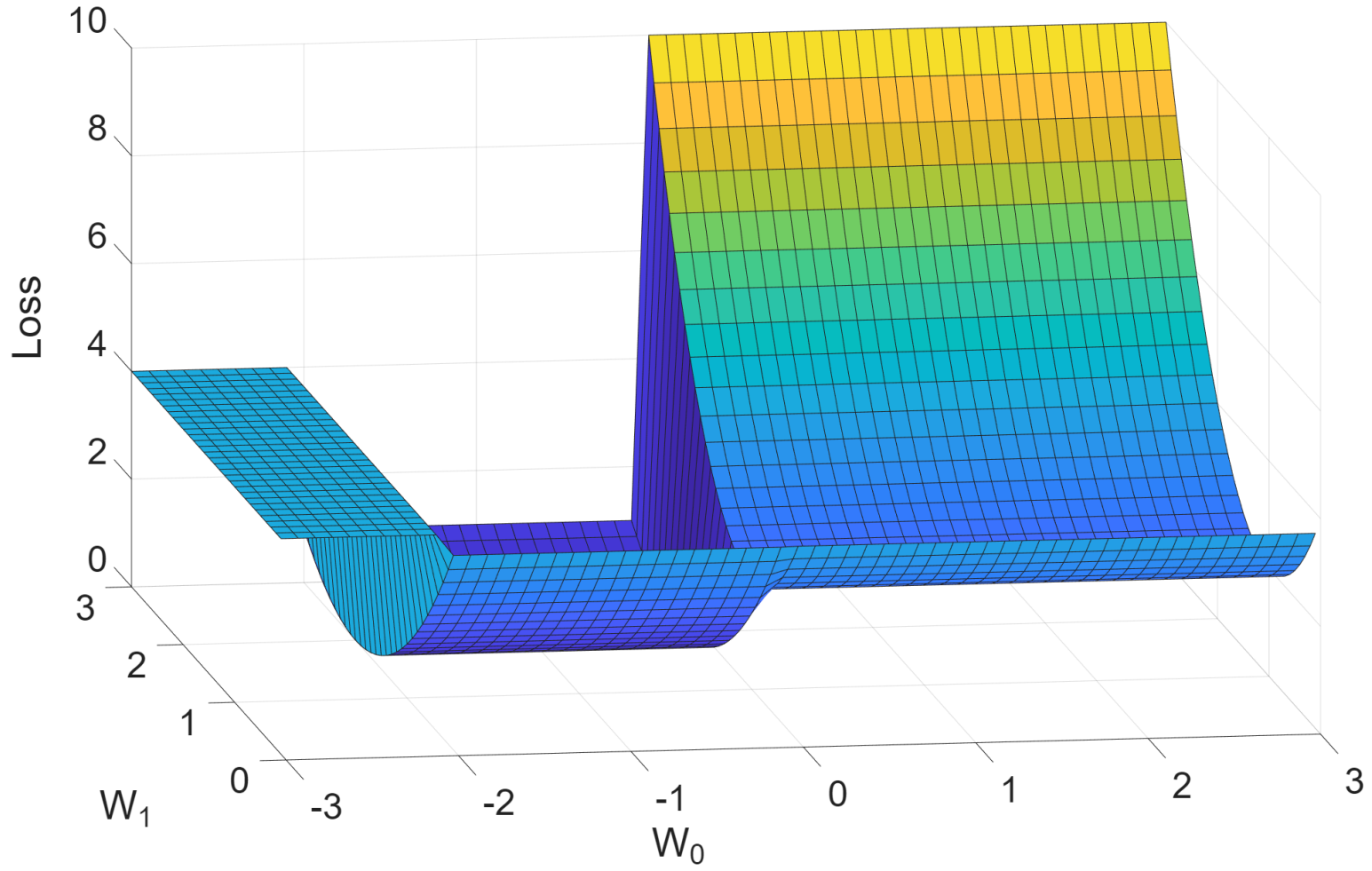


Example

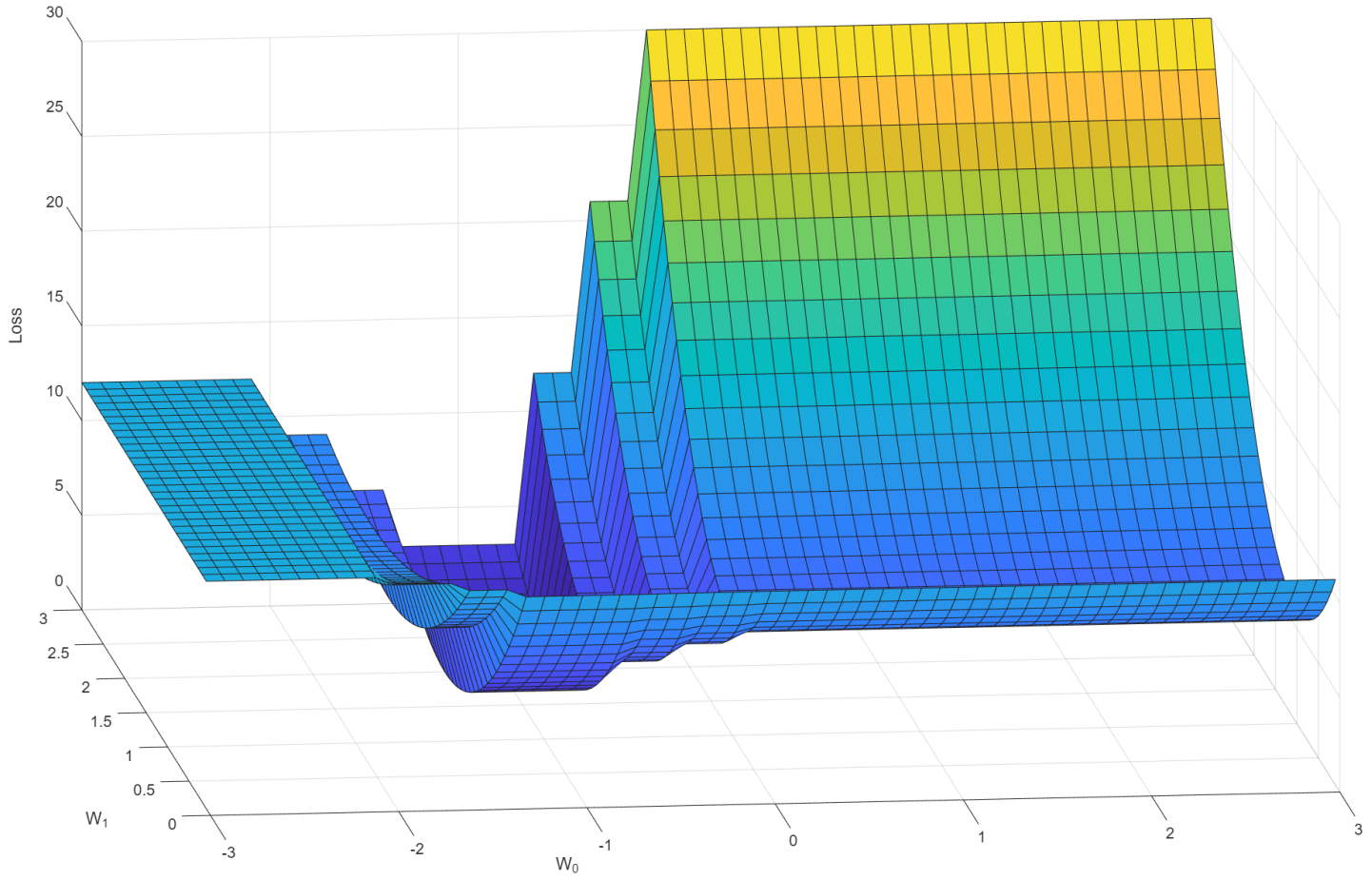
- Fit $w_1 u(x + w_0)$ with MSE loss

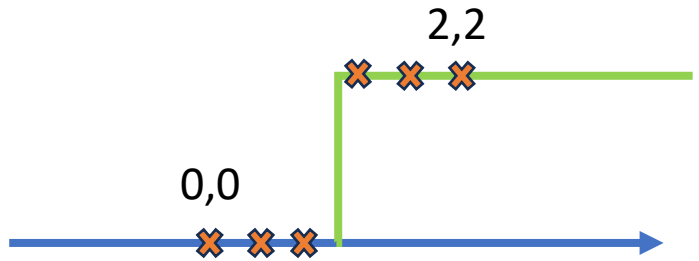


Loss Surface



Loss Surface with More Data





Challenges

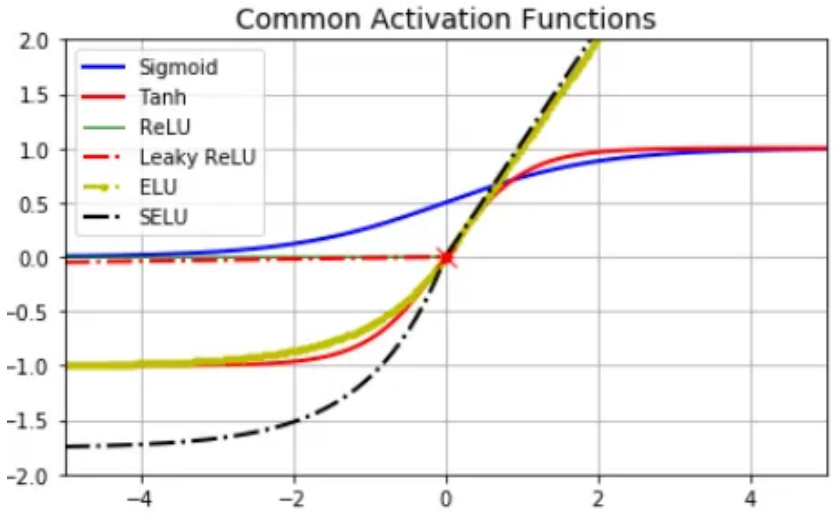
- How to find optimal values?
  - Find roots of derivative expression
- Derivatives should exist
- Roots: Move along the curve to find a flat region
  - Convex surface? Unique solution? Local minima?
  - Initialization point?

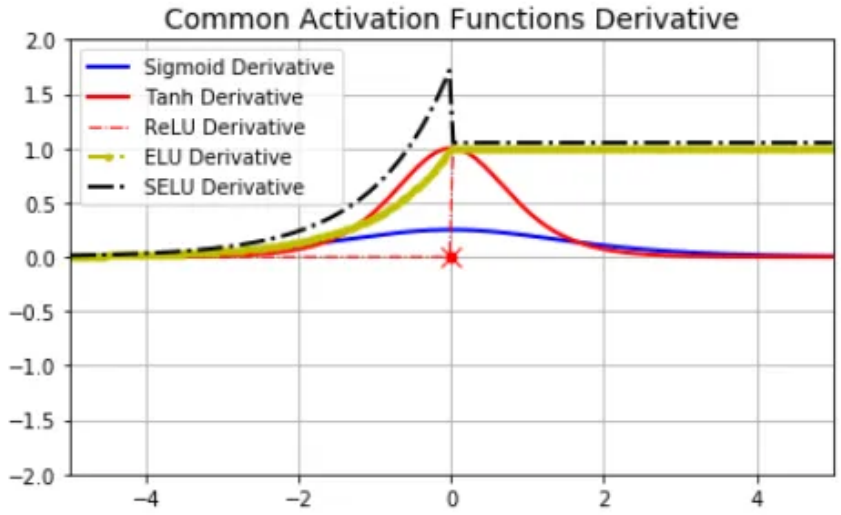
Activation Functions

- Step function has derivative = infinity at zero-input
  - Options: Sigmoid, softmax, tanh (odd symmetry)
- Non-limiting activations:

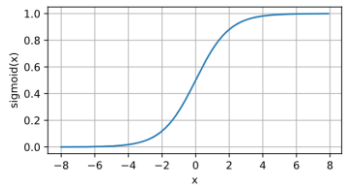
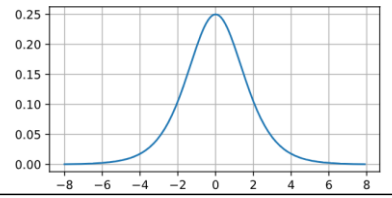
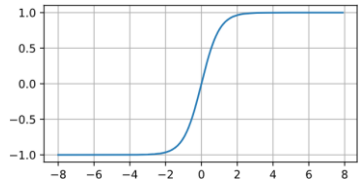
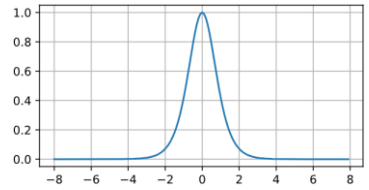
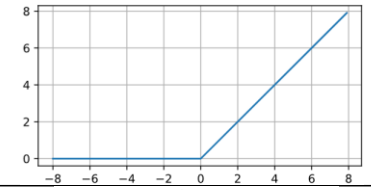
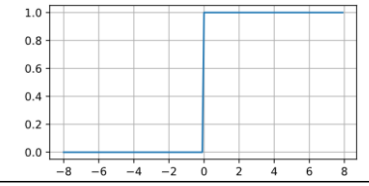
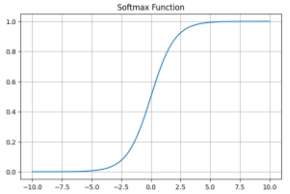
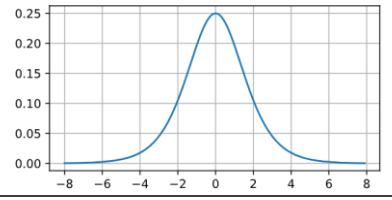
$$\text{selu}(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leqslant 0 \end{cases}$$

  - ReLU (rectified linear unit) & SeLU (scaled exponential linear unit)





Some Common Activation Functions

Function	Expression	Plot	Derivative
Sigmoid	$\frac{1}{1+\exp(-x)}$		
Tanh	$\frac{1-\exp(-2x)}{1+\exp(-2x)}$		
ReLU	$\max(x,0)$		
Softmax	$\frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$		

Backpropagation

- For input-label pair (x, y) and loss function C(.)

$$C(y, f^L(W^L f^{L-1}(W^{L-1} \cdots f^2(W^2 f^1(W^1 x)) \cdots)))$$

- The gradient is $\frac{dC}{da^L} \cdot \frac{da^L}{dz^L} \cdot \frac{dz^L}{da^{L-1}} \cdot \frac{da^{L-1}}{dz^{L-1}} \cdot \frac{dz^{L-1}}{da^{L-2}} \cdot \ldots \cdot \frac{da^1}{dz^1} \cdot \frac{\partial z^1}{\partial x}$

$$\nabla_x C = (W^1)^T \cdot (f^1)' \circ \ldots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$$

- At each layer,

$$(f^1)' \circ (W^2)^T \cdot (f^2)' \circ \cdots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$$
$$(f^2)' \circ \cdots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$$

$$(f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$$
$$(f^L)' \circ \nabla_{a^L} C,$$

Backpropagation with 2-layers

Implementation Details

- Many layers → product of many gradients
  - Exploding or vanishing
  - Activation function with smaller gradients (sigmoid) can vanish
  - Poor choice of initial point can lead to explosion
- Poor performance if all weights are initialized with the same value
  - Random (Xavier) Initialization: Gaussian with s.d.
- Early stopping
  - Stop when validation error is not reducing or reached an inflection point